

ROBOTICS SUMMER SCHOOL - FUNDAMENTALS

Multi-view geometry III

Building SfM, BA, and Lie groups one question at a time

Krish Pandya

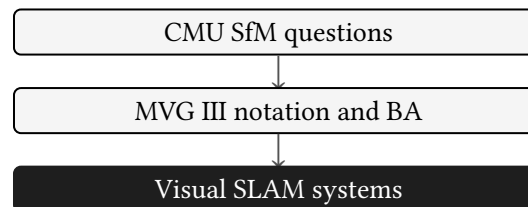
Morning session

Session map

Flow:

1. SfM motivation and multi-view setup.
2. Notation for poses, points, and projections.
3. Bundle adjustment and sparse normal equations.
4. Lie group updates for valid camera poses.
5. Bridge to visual SLAM backends.

The same projection model appears in SfM, BA, pose graphs, and visual SLAM backends.



Where the CMU slides leave us

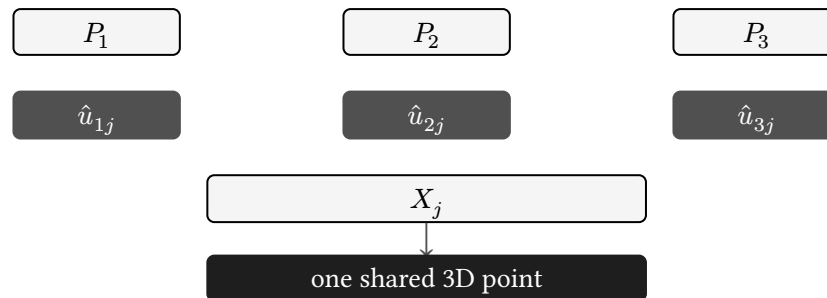
CMU has already covered:

- camera from known 3D points,
- 3D point from known cameras,
- the circular dependency,
- incremental SfM,
- BA as reprojection-error minimization.

We continue from the BA objective:

$$\min_{\{T_i\}, \{X_j\}} \sum_{(i,j) \in \mathcal{O}} \|u_{ij} - \pi(K_i T_i X_j)\|^2$$

ONE POINT, THREE CAMERAS



From here, the lecture is about how this objective is solved and why it becomes the backend of visual SLAM.

Notation checkpoint

Prediction:

$$\hat{u}_{ij} = \pi(K_i T_i X_j)$$

Residual:

$$r_{ij} = u_{ij} - \hat{u}_{ij}$$

Weighted residual:

$$\|r_{ij}\|_{\Sigma_{ij}^{-1}}^2$$

Practical BA:

$$\min \sum_{(i,j) \in \mathcal{O}} \rho\left(\|r_{ij}\|_{\Sigma_{ij}^{-1}}^2\right)$$

ρ reduces the effect of large residuals.

Why BA is nonlinear

Projection divides by depth:

$$\hat{u} = f_x \frac{X_c}{Z_c} + c_x$$

$$\hat{v} = f_y \frac{Y_c}{Z_c} + c_y$$

Pose also contains rotation, which is constrained.

BA cannot be solved once with ordinary linear least squares. We solve a sequence of local linear problems.

Linearization:

$$r(x + \delta) \approx r(x) + J\delta$$

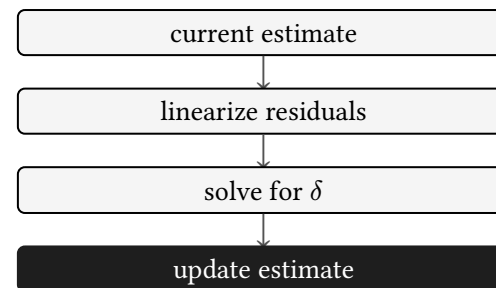
One optimization step

Local least-squares problem:

$$\min_{\delta} \|J\delta + r\|^2$$

Normal equations:

$$J^T W J \delta = -J^T W r$$



Why the matrix is sparse

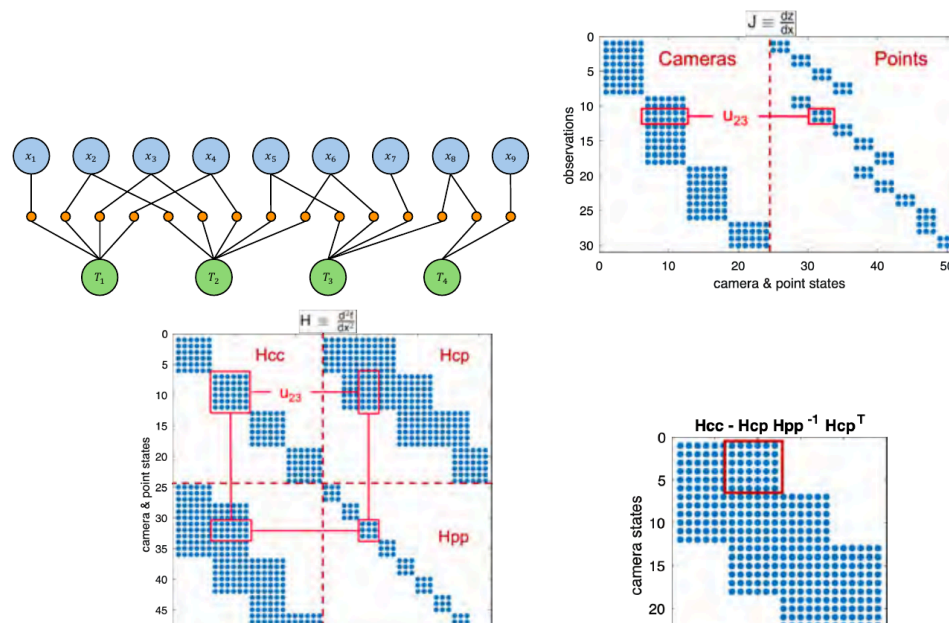
One residual touches only:

- camera pose T_i ,
- point X_j ,
- optional intrinsics K_i .

It does not depend on every other camera or point.

214

Visual SLAM



This sparse structure makes large-scale BA possible.

Schur complement idea

Split increments into cameras and points:

$$\begin{pmatrix} H_{cc} & H_{cp} \\ H_{pc} & H_{pp} \end{pmatrix} \begin{pmatrix} \delta_c \\ \delta_p \end{pmatrix} = \begin{pmatrix} b_c \\ b_p \end{pmatrix}$$

Since points are independent after cameras are fixed, H_{pp} is block diagonal.

Eliminate points and solve cameras first:

$$(H_{cc} - H_{cp}H_{pp}^{-1}H_{pc})\delta_c = b'$$

BA solvers exploit this camera-point structure instead of solving one dense system.

If all distances are multiplied by 2, does a monocular image change?

Gauge freedom

Monocular projection cannot see absolute scale:

$$X_j \rightarrow sX_j$$

$$t_i \rightarrow st_i$$

The image projection stays the same.

Fixes:

- fix the first pose,
- add a scale prior,
- use stereo,
- use IMU,
- use known object size.

Without a gauge choice, the Hessian is singular.

Why do we need Lie groups if BA is just least squares?

Rotations are constrained

Rotation matrices must satisfy:

$$\begin{aligned}R^T R &= I \\ \det(R) &= 1\end{aligned}$$

If we add arbitrary numbers to R , it usually stops being a rotation.

Rigid poses live in $\text{SE}(3)$:

$$T = \begin{pmatrix} R & t \\ 0 & 1 \end{pmatrix}$$

The optimizer needs vector updates, but the pose must stay a valid rigid transform.

Tangent-space updates

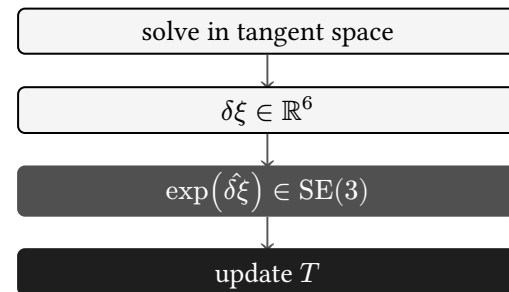
The optimizer solves for a small 6D increment:

$$\delta\xi = \begin{pmatrix} \delta\omega \\ \delta v \end{pmatrix} \in \mathbb{R}^6$$

Convert the increment to a valid pose update:

$$T \leftarrow \exp(\hat{\delta\xi})T$$

Here $\hat{\delta\xi} \in \mathfrak{se}(3)$ and $\exp(\hat{\delta\xi}) \in \text{SE}(3)$.



The hat operator

For $\omega = (\omega_x \ \omega_y \ \omega_z)^T$:

$$\hat{\omega} = \begin{pmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{pmatrix}$$

$$\hat{\omega}p = \omega \times p$$

For $\text{SE}(3)$:

$$\hat{\xi} = \begin{pmatrix} \hat{\omega} & v \\ 0 & 0 \end{pmatrix}$$

Maps:

$$\text{Exp} : \text{se}(3) \rightarrow \text{SE}(3)$$

$$\text{Log} : \text{SE}(3) \rightarrow \text{se}(3)$$

Matrix derivative starter

Use column-vector convention:

$$x \in \mathbb{R}^n, \quad a \in \mathbb{R}^n$$

Scalar dot product:

$$s = a^T x = x^T a$$
$$\nabla_x s = a$$

Row-vector notation:

$$s = ax^T \quad \Rightarrow \quad \frac{\partial s}{\partial x} = a$$

Linear vector map:

$$y = Ax + b$$
$$\frac{\partial y}{\partial x} = A$$

Component check:

$$y_i = \sum_j A_{ij} x_j + b_i$$
$$\frac{\partial y_i}{\partial x_j} = A_{ij}$$

Quadratic scalar:

$$q = x^T A x$$
$$\nabla_x q = (A + A^T)x$$

If A is symmetric, then $\nabla_x (x^T A x) = 2Ax$.

Least-squares derivative

Start with a weighted linear residual:

$$e(x) = Ax - b$$

$$E(x) = e(x)^T W e(x)$$

Substitute:

$$E(x) = (Ax - b)^T W (Ax - b)$$

Gradient:

$$\nabla_x E = 2A^T W (Ax - b)$$

For BA, x becomes δ , A becomes J , and $-b$ becomes r :

$$e(\delta) = J\delta + r$$

$$\nabla_\delta E = 2J^T W (J\delta + r)$$

Set gradient to zero:

$$J^T W J \delta = -J^T W r$$

Lie groups inside BA

Camera-frame point:

$$p_c = T_i X_j$$

A left pose perturbation changes it by:

$$\delta p_c \approx (-\hat{p}_c \ I_3) \delta \xi$$

This enters the pose Jacobian for reprojection error.

With residual $r = u - \pi(p_c)$:

$$J_{\text{pose}} = \frac{\partial r}{\partial \delta \xi} = -\frac{\partial \pi}{\partial p_c} (-\hat{p}_c \ I_3)$$

Lie algebra is not extra theory. It is how BA code updates camera poses.

From BA to a pose graph

206

Visual SLAM

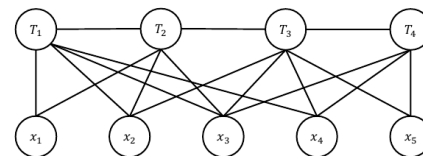
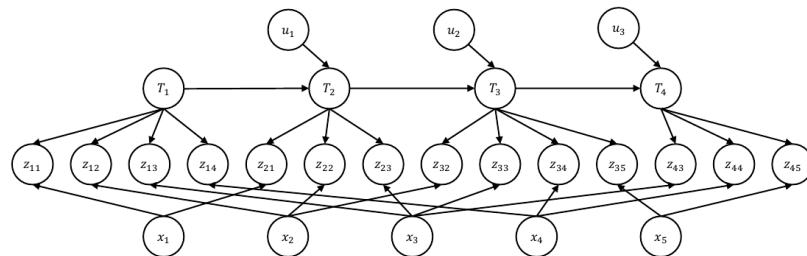
BA state:

$$\{T_i\}, \{X_j\}$$

BA residual:

$$u_{ij} - \pi(K_i T_i X_j)$$

After tracking or local BA, the frontend can summarize motion between keyframes as a relative pose measurement Z_{ij} .



The graph view keeps pose nodes and observation constraints. A pose-graph backend keeps only pose nodes and pose-pose edges.

Pose-only graph

Pose-graph state:

$$\{T_i\}$$

Pose-graph measurement:

$$Z_{ij} \approx T_i^{-1}T_j$$

Pose-graph residual:

$$\text{Log}(Z_{ij}^{-1}T_i^{-1}T_j)$$

A pose graph is what remains when the map points are removed and each edge stores a measured transform between two poses.

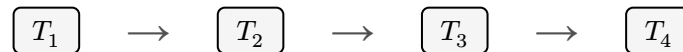
Pose graph optimization

A pose graph keeps camera poses as nodes and relative-pose measurements as edges.

$$Z_{ij} \approx T_i^{-1} T_j$$

Example edges:

- odometry: Z_{12}, Z_{23}, Z_{34} ,
- loop closure: Z_{14} .



The loop edge says the path from T_1 to T_4 should agree with a newly recognized return to the same place.

Small pose-graph example

In one dimension, fix $x_1 = 0$.

Odometry edges:

$$z_{12} = 1, \quad z_{23} = 1, \quad z_{34} = 1$$

Odometry alone predicts:

$$x_4 = 3$$

A loop detector says frame 4 returned near frame 1:

$$z_{14} = 0$$

The residuals are:

$$r_{ij} = (x_j - x_i) - z_{ij}$$

Now $x_4 = 3$ fits odometry but violates the loop edge. Optimization moves all poses to reduce total weighted error.

Pose graph residual

Predicted relative pose:

$$\hat{Z}_{ij} = T_i^{-1}T_j$$

Pose error:

$$r_{ij} = \text{Log}(Z_{ij}^{-1}T_i^{-1}T_j)$$

If the measurement and prediction agree, $r_{ij} = 0$.

Objective:

$$\min_{\{T_i\}} \sum_{(i,j) \in \mathcal{E}} r_{ij}^T \Omega_{ij} r_{ij}$$

Log maps a pose error into a vector residual that least squares can optimize.

Sim(3) and scale drift

Monocular systems can drift in scale.

A similarity transform includes scale:

$$S = \begin{pmatrix} sR & t \\ 0 & 1 \end{pmatrix} \in \text{Sim}(3)$$

Used for:

- monocular loop closure,
- map merging,
- aligning SfM to metric sensors.

SE(3) corrects pose. Sim(3) corrects pose and scale.

Bridge from SfM to visual SLAM

CMU SfM setup:

- offline,
- many passes over all images,
- global BA is possible.

Visual SLAM setup:

- online,
- bounded compute,
- local BA and pose graph updates.

Shared math:

- reprojection residuals,
- sparse least squares,
- Schur complement,
- gauge fixing,
- Lie group updates.

Visual SLAM is SfM under time, memory, tracking, and relocalization constraints.

References

LOCAL PDFS

Cornell CS5670: Structure from Motion.

Ethan Eade: Lie Groups for Computer Vision.

Dellaert and Kaess: Factor Graphs for Robot Perception.

CMU 16-385: Structure from Motion.

BOOKS AND SYSTEMS

Hartley and Zisserman: Multiple View Geometry.

COLMAP documentation.

SLAM Handbook, chapters 1, 7, 11, and 13.